

Numeri casuali

Table of contents

Numeri (pseudo) casuali	1
Numeri pseudo casuali in Python	1
Importare un modulo	2
Perché si chiamano numeri <i>pseudo</i> casuali?	2
Il concetto di <i>seed</i>	2
Funzioni principali del modulo random	3
random.randint(a, b)	3
random.random()	3
random.uniform(a, b)	4
random.shuffle(lista)	4
Riassunto	5
Esempio: squadre di basket	5
Metodo Monte Carlo	7
Idea geometrica	8
Algoritmo	9
Esempio in Python	9
Osservazioni	10
ESERCIZI	10
1. Stimare l'area sotto una curva	10
Suggerimento	11
2. Stimare pi greco con il quarto di cerchio	11
3. Stimare l'area sotto una parabola	12

Numeri (pseudo) casuali

Numeri pseudo casuali in Python

In Python è possibile generare numeri casuali utilizzando il modulo `random`.

Importare un modulo

Un **modulo** è un insieme di funzioni già pronte che Python mette a disposizione. Per usare le funzioni del modulo **random** bisogna prima importarlo:

```
import random
```

Con questa istruzione diciamo a Python di caricare il modulo **random**, così da poter utilizzare le sue funzioni scrivendo:

```
random.nome_funzione()
```

Perché si chiamano numeri *pseudo* casuali?

I numeri generati dal computer non sono veramente casuali. Vengono prodotti da un algoritmo matematico che crea una sequenza di valori che **sembra casuale**, ma che in realtà è deterministica.

Per questo motivo si parla di numeri **pseudo casuali**.

Se l'algoritmo parte dalle stesse condizioni iniziali, genererà sempre la stessa sequenza.

Il concetto di *seed*

La condizione iniziale usata dall'algoritmo si chiama **seed** (seme).

In Python si può impostare il seed con:

```
random.seed(10)
```

Usando sempre lo stesso seed si ottengono sempre gli stessi numeri (pseudo) casuali. Questo è utile, ad esempio, nei test o negli esperimenti ripetibili.

Esempio:

```
import random

random.seed(10)

print(random.randint(1, 10))
print(random.randint(1, 10))
```

Ogni volta che il programma viene eseguito, i risultati saranno identici.

Se non si imposta manualmente il seed, Python ne sceglie uno automaticamente usando informazioni come l'orario corrente.

Funzioni principali del modulo random

`random.randint(a, b)`

Genera un numero intero casuale compreso tra **a** e **b** inclusi.

```
import random  
  
n = random.randint(1, 10)  
print(n)
```

Possibili risultati:

```
3  
7  
10
```

`random.random()`

Genera un numero *float* casuale compreso tra 0.0 e 1.0.

```
import random  
  
x = random.random()  
print(x)
```

Possibile risultato:

```
0.482731
```

`random.uniform(a, b)`

Genera un numero *float* casuale compreso tra **a** e **b**.

```
import random  
  
y = random.uniform(1.5, 5.0)  
print(y)
```

Possibile risultato:

3.7264

`random.shuffle(lista)`

Mescola casualmente gli elementi di una lista.

La funzione **modifica direttamente (in place) la lista originale**. Utilizza quindi gli ‘effetti collaterali’.

```
import random  
numeri = [1, 2, 3, 4, 5]  
random.shuffle(numeri)  
print(numeri)
```

Possibili risultati:

[3, 1, 5, 2, 4]

La funzione `shuffle()` è utile, ad esempio, per:

- mescolare un mazzo di carte;
- cambiare l’ordine degli elementi in una lista;
- creare sequenze casuali.

Funzione	Descrizione
----------	-------------

Riassunto

Funzione	Descrizione
<code>random.randint(a, b)</code>	Intero casuale tra <code>a</code> e <code>b</code>
<code>random.random()</code>	Float casuale tra 0.0 e 1.0
<code>random.uniform(a, b)</code>	Float casuale tra <code>a</code> e <code>b</code>
<code>random.seed(x)</code>	Imposta il seed della sequenza pseudo casuale
<code>random.shuffle(numeri)</code>	Mescola la lista 'in place'

Esempio: squadre di basket

Due amici decidono di andare a giocare a basket. Ogni squadra è composta da sei giocatori (5 + riserva). I giocatori vengono assegnati alla squadra A o B in maniera del tutto casuale.

Qual è la probabilità che i due amici vengano assegnati alla stessa squadra?

- Rappresento i giocatori con i numeri da 1 a 12
- Rappresento i due amici con i numeri 1 e 2
- Rappresento le squadre con le liste A e B
- Assegno casualmente i giocatori alle squadre A e B
- Verifico se sono capitati nella stessa squadra

Ripetendo la simulazione molte volte, ho una stima 'frequentista' della probabilità che i due amici vengano assegnati alla stessa squadra.

Posso generalizzare il problema scrivendo una funzione `insieme(n)` che accetta il numero totale di giocatori (in questo caso 12).

La funzione rende 1 se i giocatori sono nella stessa squadra, altrimenti rende 0.

Ripeto la simulazione molte volte e conto i valori 1.

La logica della funzione `insieme(n)` è la seguente: per ogni giocatore,

- se la squadra A è già al completo, viene assegnato alla squadra B (o viceversa)
 - altrimenti viene selezionata una squadra in modo casuale.
-

```
import random

def insieme(n):
    """ rende 1 se i giocatori 1, 2 sono nella stessa squadra"""
    s1=[]
    s2=[]

    giocatori= list(range(1,n+1))
    # for g in range(1,n+1))
    for g in giocatori:
        if len(s1) == n/2:
            s2.append(g)
        elif len(s2) == n/2:
            s1.append(g)
        else:
            # tiro la moneta
            v=random.randint(1,2)
            if v == 1:
                s1.append(g)
            else:
                s2.append(g)
    # if (1 in s1)== (2 in s1)
    if ((1 in s1) and (2 in s1) ) or ((1 in s2) and (2 in s2) ):
        return 1
    else:
        return 0

# -----
insieme(12)
```

Attenzione, la funzione contiene un errore sottile. La stima è errata. Riuscite a trovarlo?

Uso della funzione

```
n_prove = 10000
conta = 0
for i in range(n_prove):
    v = insieme(12)
    conta=conta+v
prb = conta/n_prove
print(prb)
```

Spiegazione dell'errore.

Supponiamo che il primo amico venga assegnato alla squadra A. A quel punto la squadra A ha già 1 posto occupato, e restano 11 giocatori da assegnare, tra cui il secondo amico.

Nella squadra A restano solo 5 posti liberi, nella squadra B ne restano 6. La situazione è asimmetrica perché, dopo aver collocato il primo amico, le due squadre non sono più equivalenti rispetto al secondo amico.

Quindi è leggermente meno probabile che il secondo amico finisca nella squadra del primo amico ($p = \frac{5}{11}$).

A livello di codice l'errore sta nel fatto che il partecipante 1 è sempre scelto primo, quindi è l'unico a vedere sempre una situazione simmetrica. Dobbiamo rimescolare *ogni volta* la lista dei giocatori usando

```
random.shuffle(giocatori)
```

- Esercizio

Riscrivere la funzione `insieme()` usando `shuffle` e lo slicing, senza usare i cicli.

Metodo Monte Carlo

Stima dell'area di un cerchio.

Il metodo di Monte Carlo è una tecnica numerica basata sull'uso di numeri casuali per stimare grandezze matematiche.

Nel caso dell'area di un cerchio, l'idea consiste nel generare molti punti casuali in una regione nota e verificare quanti punti cadono all'interno del cerchio.

Consideriamo un cerchio di raggio r , centrato nell'origine $(0,0)$.

L'equazione del cerchio è:

$$x^2 + y^2 = r^2$$

Idea geometrica

Il cerchio può essere inscritto in un quadrato di lato $2r$.

- area del quadrato:

$$A_{\text{quadrato}} = (2r)^2 = 4r^2$$

- area del cerchio:

$$A_{\text{cerchio}} = \pi r^2$$

Se generiamo molti punti casuali uniformemente distribuiti nel quadrato:

- alcuni cadranno dentro il cerchio;
- altri cadranno fuori.

Il rapporto $\text{punti_interni} / \text{punti_totali}$ approssima il rapporto tra le aree

$$A_{\text{cerchio}} / A_{\text{quadrato}} = \pi r^2 / 4r^2 = \pi / 4$$

Da cui:

$$A_{\text{cerchio}} = 4r^2 * (\text{punti_interni} / \text{punti_totali})$$

Algoritmo

1. Generare N punti casuali nel quadrato $[-r, r] \times [-r, r]$
2. Per ogni punto (x,y) verificare se:

$$x^2 + y^2 \leq r^2$$

3. Contare quanti punti cadono nel cerchio
 4. Stimare l'area usando il rapporto tra punti interni e punti totali
-

Esempio in Python

```
import random

def stima_area_cerchio(r, N = 100000 ):
    # contatore punti interni al cerchio
    interni = 0

    for _ in range(N):

        # coordinate casuali nel quadrato
        x = random.uniform(-r, r)
        y = random.uniform(-r, r)

        # verifica appartenenza al cerchio
        if x**2 + y**2 <= r**2:
            interni += 1

    # stima area del cerchio
    area_stimata = 4 * r**2 * interni / N
    return area_stimata

#----
# raggio del cerchio
r = 1
area_stimata = stima_area_cerchio(r)
print("Area stimata:", area_stimata)
```

Osservazioni

- aumentando il numero di punti casuali N , la stima tende a migliorare;
- il metodo è probabilistico: esecuzioni diverse producono risultati leggermente diversi;
- scegliendo $r = 1$, la stima dell'area coincide con una stima di π , perché:

$$A = \pi r^2 = \pi$$

ESERCIZI

L'idea generale è sempre la stessa:

1. si genera un grande numero di punti casuali;
 2. si conta quanti punti soddisfano una certa condizione;
 3. si usa la proporzione ottenuta per stimare un'area, un volume, una probabilità o un integrale.
-

1. Stimare l'area sotto una curva

Stimare l'area della regione compresa tra l'asse delle ascisse e la funzione:

$$f(x) = \sin(x) + 2$$

nell'intervallo:

$$0 \leq x \leq 4$$

La regione è quindi:

$$0 \leq y \leq \sin(x) + 2$$

Suggerimento

Generare punti casuali nel rettangolo:

$$0 \leq x \leq 4$$

$$0 \leq y \leq 3$$

Il rettangolo ha area:

$$A_{rettangolo} = 4 \cdot 3 = 12$$

Contare quanti punti cadono sotto la curva, cioè quanti soddisfano:

$$y \leq \sin(x) + 2$$

La stima dell'area sarà:

$$A \approx A_{rettangolo} \cdot \frac{\text{punti sotto la curva}}{\text{punti totali}}$$

2. Stimare pi greco con il quarto di cerchio

Nel quadrato unitario:

$$0 \leq x \leq 1$$

$$0 \leq y \leq 1$$

si considera il quarto di cerchio di raggio 1 centrato nell'origine.

Un punto appartiene al quarto di cerchio se:

$$x^2 + y^2 \leq 1$$

L'area del quadrato è:

$$A_{quadrato} = 1$$

L'area del quarto di cerchio è:

$$A_{\text{quarto cerchio}} = \frac{\pi}{4}$$

Quindi:

$$\frac{\text{punti nel quarto di cerchio}}{\text{punti totali}} \approx \frac{\pi}{4}$$

Da cui:

$$\pi \approx 4 \cdot \frac{\text{punti nel quarto di cerchio}}{\text{punti totali}}$$

3. Stimare l'area sotto una parabola

Stimare l'integrale:

$$\int_0^1 x^2 dx$$

usando Monte Carlo.

Si genera un punto casuale nel quadrato unitario:

$$0 \leq x \leq 1$$

$$0 \leq y \leq 1$$

Il punto è sotto la parabola se:

$$y \leq x^2$$

Poiché l'area del quadrato è 1, la stima dell'integrale è:

$$\int_0^1 x^2 dx \approx \frac{\text{punti sotto la parabola}}{\text{punti totali}}$$

Il valore esatto è:

$$\int_0^1 x^2 dx = \frac{1}{3}$$